

Lecture 3: Conditional Statements and Loops

Lecture Contents

- Control Structures
 - Sequence Structures
 - **Decision Structures**
 - if Statement
 - if else Statement
 - elif Statement (Chained Conditionals)
 - Nested Conditionals
 - **Repetition Structures**
 - Loops
 - for loop
 - while loop

Control Structure

- A control structure controls the order in which a set of statements execute
 - **Sequence Structure**
 - All statements are executed in a sequential order
 - Sequence structure cannot handle all types of real word scenarios
 - **Decision Structure / Conditional Statements**
 - Allow to execute different statements based on a condition
 - If condition is True execute statement 1 otherwise execute statement 2
 - Examples / Types
 - if Statement
 - if-else Statement
 - elif Statement
 - **Repetition Structures / Loops**
 - Execute a block of code specified number of times
 - for loop and while loop

Decision Structure : Conditional Statements

- Conditional Statement
 - A line of code that includes a Boolean expression
 - Evaluates to True or False
 - Includes Relational Operators

Relational operators

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

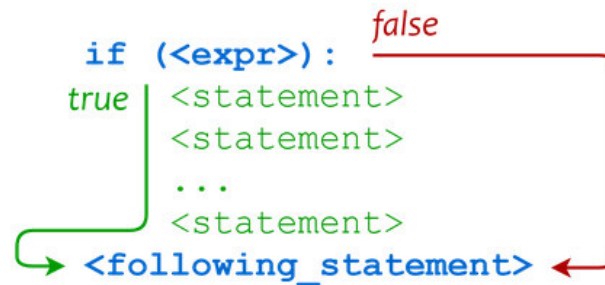
Arithmetic Operators

Table 2-3 Python math operators

Symbol	Operation	Description
+	Addition	Adds two numbers
-	Subtraction	Subtracts one number from another
*	Multiplication	Multiplies one number by another
/	Division	Divides one number by another and gives the result as a floating-point number
//	Integer division	Divides one number by another and gives the result as an integer
%	Remainder	Divides one number by another and gives the remainder
**	Exponent	Raises a number to a power

if Statement (Lab Task 1.1)

- if statement is used to execute a block of code based on a Boolean expression
- Syntax:



- Things to Remember :
 - **if** is in lower case
 - There is a **colon** after the Boolean expression
 - Body of **if** should be indented
 - Can contain a single or multiple statements in the body

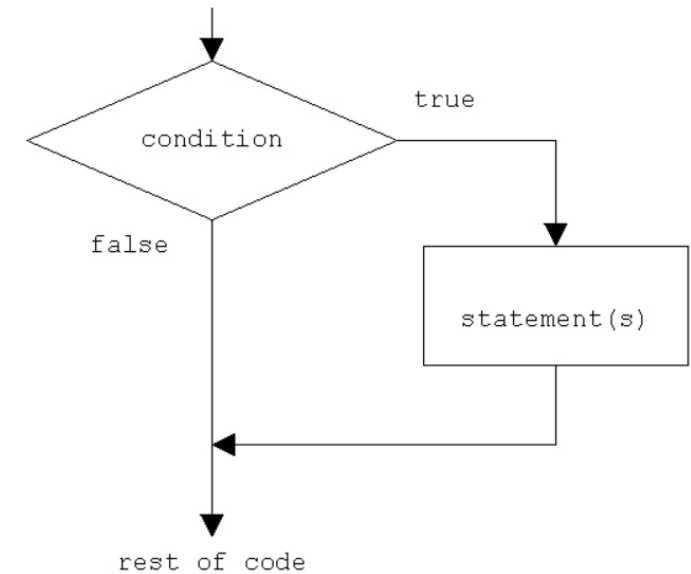
if Statement Example / Flow Chart

```
a=100 # Assignemnt Statement
b=20  # Assignment Statement
if(a>b): # Boolean Expression
    print("a is greater than b")      # if body - statement 1
    print("a= {} b={} ".format(a,b)) # if body - statement 2
print("This print statement will execute no matter a>b or a<b")
```

a is greater than b
a= 100 b=20
This print statement will execute no matter a>b or a<b

```
a=100 # Assignemnt Statement
b=200 # Assignment Statement
if(a>b): # Boolean Expression
    print("a is greater than b")      # if body - statement 1
    print("a= {} b={} ".format(a,b)) # if body - statement 2
print("This print statement will execute no matter a>b or a<b")
```

This print statement will execute no matter a>b or a<b



Problem Solving

Task1.1 : Using if Statement

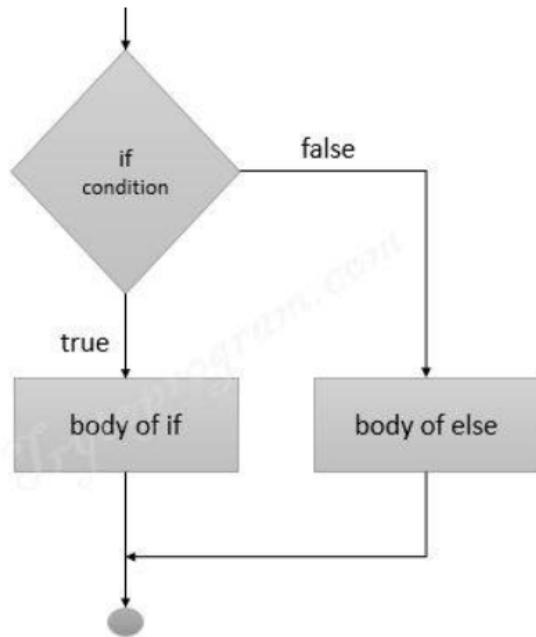
Using the if Statement

Kathryn teaches a science class and her students are required to take three tests. She wants to write a program that her students can use to calculate their average test score. She also wants the program to congratulate the student enthusiastically if the average is greater than 95. Here is the algorithm in pseudocode:

```
Get the first test score  
Get the second test score  
Get the third test score  
Calculate the average  
Display the average  
If the average is greater than 95:  
    Congratulate the user
```


if else Statement (Lab Task 1.2)

- Sometimes we need to execute a specified block of code when condition is true and another block of code when condition is false



```
a = 33
b = 200
if b > a:
    print("b is greater than a")
else:
    print("a is greater than or equal to b")
```

b is greater than a

```
a = 330
b = 200
if b > a:
    print("b is greater than a")
else:
    print("a is greater than or equal to b")
```

a is greater than or equal to b

Problem Solving

Task 1.2: Using if else Statement

Using the if-else Statement

Chris owns an auto repair business and has several employees. If any employee works over 40 hours in a week, he pays them 1.5 times their regular hourly pay rate for all hours over 40. He has asked you to design a simple payroll program that calculates an employee's gross pay, including any overtime wages. You design the following algorithm:

Get the number of hours worked.

Get the hourly pay rate.

If the employee worked more than 40 hours:

Calculate and display the gross pay with overtime.

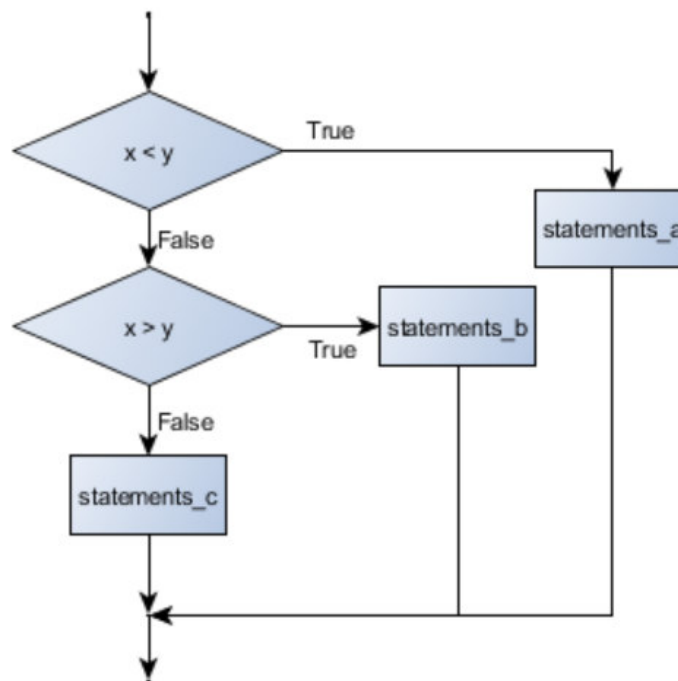
Else:

Calculate and display the gross pay as usual.

Chained Conditionals – elif

- Sometimes there are more than two possibilities .

```
if x < y:  
    STATEMENTS_A  
elif x > y:  
    STATEMENTS_B  
else:  
    STATEMENTS_C
```



```
x=60  
y=50  
if(x<y):  
    print(" x is less than y")  
elif(x>y):  
    print("x is greater than y")  
else:  
    print(" x and y are equal")
```

x is greater than y

Problem Solving

Task 1.3 : Using elif Statement

Multiple Nested Decision Structures

Dr. Suarez teaches a literature class and uses the following 10-point grading scale for all of his exams:

Test Score	Grade
90 and above	A
80–89	B
70–79	C
60–69	D
Below 60	F

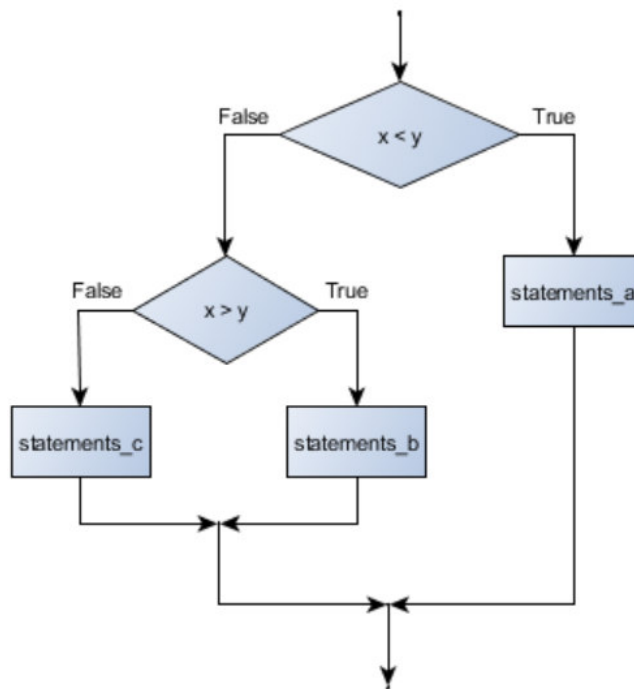
He has asked you to write a program that will allow a student to enter a test score and then display the grade for that score. Here is the algorithm that you will use:

1. Ask the user to enter a test score.
2. Determine the grade

Nested Conditional Statements (Lab Task 1.3)

- An if statement or an if-else statement can be nested within another if or else block

```
if x < y:  
    STATEMENTS_A  
else:  
    if x > y:  
        STATEMENTS_B  
    else:  
        STATEMENTS_C
```



```
x = 30  
y = 20  
if x < y:  
    print("Statement A: x is less than y")  
else:  
    if x > y:  
        print(" Statement B: x is greater than y")  
    else:  
        print("Statement C: x is equal to y")
```

Statement B: x is greater than y

Checking Multiple Conditions

- Logical Operators can be used to combine multiple conditions

Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	<code>x < 5 and x < 10</code>
or	Returns True if one of the statements is true	<code>x < 5 or x < 4</code>
not	Reverse the result, returns False if the result is true	<code>not(x < 5 and x < 10)</code>

Multiple Conditions – Examples

```
student='Yes'
age=25
gender='Male'
if student=='Yes'and age>20 and gender=='Male':
    print("This person is likely to buy your product")
else:
    print("This person is not likely to buy your product")
```

This person is likely to buy your product

```
student='Yes'
age=25
gender='Male'
if student=='Yes'and age>20 or gender=='Female':
    print("This person is likely to buy your product")
else:
    print("This person is not likely to buy your product")
```

This person is likely to buy your product

Switch Case Statement

- Languages like C , C++ , Java and Others have Switch–Case statement that can be used as an alternative to elif statement to check multiple conditions.
- Earlier versions of python do not have any switch-case functionality
 - [The same can be implemented with the help of dictionaries and classes](#) and functions
- Python version 3.10 and later have **match-case** functionality

match – case (Syntax)

```
match term:
    case pattern-1:
        action-1
    case pattern-2:
        action-2
    case pattern-3:
        action-3
    case _:
        action-default
```

match-case (Example)

```
lang = input("What's the programming language you want to learn? ")

match lang:
    case "JavaScript":
        print("You can become a web developer.")

    case "Python":
        print("You can become a Data Scientist")

    case "PHP":
        print("You can become a backend developer")

    case "Solidity":
        print("You can become a Blockchain developer")

    case "Java":
        print("You can become a mobile app developer")

    case _:
        print("The language doesn't matter, what matters is solving problems.")
```

Problem Solving

Task 1.4: Time Calculator

Time Calculator

Write a program that asks the user to enter a number of seconds and works as follows:

- There are 60 seconds in a minute. If the number of seconds entered by the user is greater than or equal to 60, the program should display the number of minutes in that many seconds.
- There are 3,600 seconds in an hour. If the number of seconds entered by the user is greater than or equal to 3,600, the program should display the number of hours in that many seconds.
- There are 86,400 seconds in a day. If the number of seconds entered by the user is greater than or equal to 86,400, the program should display the number of days in that many seconds.

Repetition Structures / Loops

Repetition Structure

- A repetition structure causes a statement or group of statements to execute repeatedly
- Two Types
 - Counter Controlled Loops
 - Number of iterations is known
 - A Counter variable is used to count the number of iterations
 - **for** loop is a counter-controlled loop
 - Example: Print first 10 odd numbers
 - Condition Controlled Loops
 - Number of iterations is not known in advanced
 - Before each iteration a condition is checked
 - **while** loop is a condition-controlled loop
 - For example : play a game until choice == 'No'

for loop

- A counter-controlled loop
- Can be used to process each element in a sequence
 - Strings
 - List
 - Dictionaries
 - Sets and Tuples
- Syntax

```
for LOOP_VARIABLE in SEQUENCE:  
    STATEMENTS
```

for loop – Examples

```
mylist = ["item1", "item2", "item3"]
for x in mylist:
    print(x)
```

item1
item2
item3

```
for x in range(1,15,2):
    print(x)
```

1
3
5
7
9
11
13

```
###Looping Through a String
mystring = "Pakistan"
for j in mystring:
    print(j)
```

P
a
k
i
s
t
a
n

range() – a useful built-in function

- range()
 - Creates a type of object called an iterable
- iterable
 - An object that contains a sequence of values that can be iterated over
 - Used in counter-controlled loops (for loop for example)
- Syntax
 - range(start, stop, step)
 - By default , **start**=0 . It is an optional argument
 - **stop** is required . Specifies the end point. Itself not included
 - **step** specifies the increment. By default, it is 1

range() – syntax (Lab Task 2.1)

How to use Python range(start, stop, step)

range() returns the immutable sequence of numbers starting from the given **start integer to the stop-step**. Each number is incremented by adding step value to its preceding number

range(0, 6, 1) →

0	1	2	3	4	5
---	---	---	---	---	---

Step. (Optional) Specify the increment.
Default is **1**

Stop. (Required) specifying at which
position to stop. Not part of the result

Start. (Optional) Start number of
sequence. Default is **0**

```
for i in range(6):  
    print(i)
```

It returns a range object not list
`type(range(6)) -> class 'range'`

Nested Loops

```
Algorithm = ["Naive Bayes", "Logistic Regression", "KNN"]
Normalization = ["One-Hot", "Label", "Mannual"]
Datasets=['Telecom','Finance']
for x in Algorithm:
    for y in Normalization:
        for z in Datasets:
            print(x, y , z)
```

```
Naive Bayes One-Hot Telecom
Naive Bayes One-Hot Finance
Naive Bayes Label Telecom
Naive Bayes Label Finance
Naive Bayes Mannual Telecom
Naive Bayes Mannual Finance
Logistic Regression One-Hot Telecom
Logistic Regression One-Hot Finance
Logistic Regression Label Telecom
Logistic Regression Label Finance
Logistic Regression Mannual Telecom
Logistic Regression Mannual Finance
KNN One-Hot Telecom
KNN One-Hot Finance
KNN Label Telecom
KNN Label Finance
KNN Mannual Telecom
KNN Mannual Finance
```

Nested for Loops

```
p=[1,2,3]
d=[1,2]
q=[2,4,6]
for x in p:
    for y in d:
        for z in q:
            print("ARIMA({} , {}, {})".format(x,y,z))
```

```
ARIMA(1 ,1, 2)
ARIMA(1 ,1, 4)
ARIMA(1 ,1, 6)
ARIMA(1 ,2, 2)
ARIMA(1 ,2, 4)
ARIMA(1 ,2, 6)
ARIMA(2 ,1, 2)
ARIMA(2 ,1, 4)
ARIMA(2 ,1, 6)
ARIMA(2 ,2, 2)
ARIMA(2 ,2, 4)
ARIMA(2 ,2, 6)
ARIMA(3 ,1, 2)
ARIMA(3 ,1, 4)
ARIMA(3 ,1, 6)
ARIMA(3 ,2, 2)
ARIMA(3 ,2, 4)
ARIMA(3 ,2, 6)
```

HCF of Two Numbers

```
x=int(input("Enter the value of X :"))
y=int(input("Enter the value of X :"))
if x<y:
    smaller=x
else:
    smaller=y
for i in range(2,smaller+1):
    if((x%i==0)and(y%i==0)):
        hcf=i
print("HCF of {} and {} = {}".format(x,y,hcf))
```

Enter the value of X :54

Enter the value of X :24

HCF of 54 and 24 = 6

Print this pattern using for loops (Nested Loops)

```
*  $  $  $  $  $  $  $  $  $
*  *  $  $  $  $  $  $  $  $
*  *  *  $  $  $  $  $  $  $
*  *  *  *  $  $  $  $  $  $
*  *  *  *  *  $  $  $  $  $
*  *  *  *  *  *  $  $  $  $
*  *  *  *  *  *  *  $  $  $
*  *  *  *  *  *  *  *  $  $
*  *  *  *  *  *  *  *  *  $
*  *  *  *  *  *  *  *  *  *
```

while loop

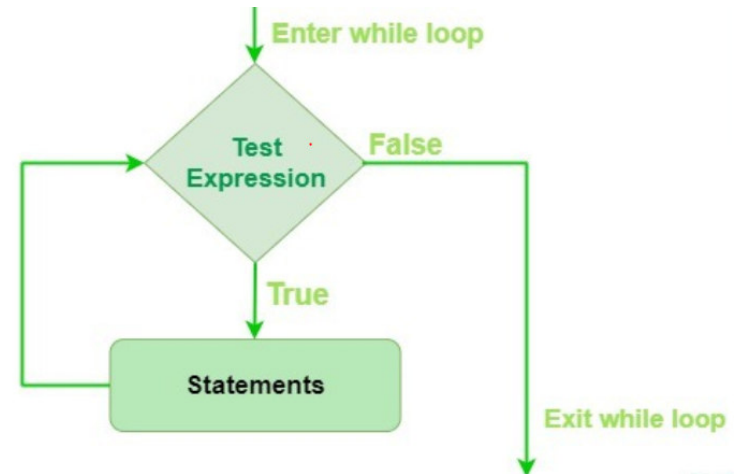
- While loop is used when number of iterations is not known
- Number of iterations depends on some condition
- Syntax:

```
while ( TEST_CONDITION):  
    STATEMENT1  
    STATEMENT2
```

- Example

```
secret=42  
guess=None  
while guess!=secret:  
    guess=int(input("Guess the number!"))
```

```
Guess the number!45  
Guess the number!78  
Guess the number!32  
Guess the number!42
```



Game: Guess the number (Lab Task 2)

- Requirements

- A secret number
- Ask the user to guess the number
- Check if the number matches
 - If number is greater – display “ You are too high”
 - If the number is less – display “ You are too low”
- Terminate when the number matches with a congratulations message

Which one is better? for or while loop

- Understandability
 - for loop is easy to understand and implement as compared to while loop
- Potential for infinite looping
 - Less chances in for loop
 - Can run into infinite iterations based on the condition
- Execution time
 - Both have initialization , test condition and increment expression
 - Actual execution time depends on the logic to be implemented
- Any for loop task can be implemented using a while loop but not vice versa

Loop Control Statements

- Loop control statements change execution from its normal sequence.

Sr.No.	Control Statement & Description
1	break statement Terminates the loop statement and transfers execution to the statement immediately following the loop.
2	continue statement Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
3	pass statement The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

Continue Statement – Examples

P
y
t
h
o
n

```
for var in "Python for Data Science":  
    if(var=='a'):  
        continue  
    print(var)
```

f
o
r

D
t

S
c
i
e
n
c
e

```
for i in range(10):  
    if i % 2 == 0:  
        continue # Skip even numbers  
    print(i)
```

Pass Statement

- The pass statement in Python is a null statement. It does not do anything. It is used as a placeholder for future code.
- For example, you might use the pass statement to define a function that you have not yet implemented, or to create a loop that you will fill in later.
- The pass statement is also used to avoid errors in situations where a statement is required syntactically, but you do not want any code to execute.
- For example, you might use the pass statement in an if statement where you do not want to take any action if the condition is met.

pass statement

```
marks = int(input("Enter the marks: "))

if marks < 60:
    print("Meet in my office")
else:
    pass # No specific action needed for marks >= 60
```

break statement-examples

break statement

Terminates the loop statement and transfers execution to the statement immediately following the loop.

```
secret=42
guess=None
while guess!=secret:
    guess=int(input("Guess the number:"))
    if guess==secret:
        print("Congratulations! You guessed it right")
    else:
        print("Wrong Guess:Try it again !")
        res=input("Do you want to continue?")
        if res.lower()=='y':
            pass
        else:
            print(" I hope you enjoyed it. See you again soon!")
            break
```

pass and continue

Key Differences:

Feature	<code>continue</code>	<code>pass</code>
Action	Skips the rest of the current iteration	Does nothing
Loop Control	Jumps to the next iteration	Continues to the next statement
Use Cases	Skipping specific iterations based on conditions	Placeholder for future code, empty blocks

pass Statement - Example

```
def my_function():  
    pass
```

This function does not do anything. It is simply a placeholder for future code.

The pass statement is a useful tool for avoiding errors and for creating placeholders for future code.

```
def divide(x, y):  
    try:  
        return x / y  
    except ZeroDivisionError:  
        pass  
  
print(divide(10, 0))
```

while True Loop

- The loop will run at least once
- A break statement is included to quit the loop based on a condition
- If there is no break statement , the loop will continue indefinitely

```
secret=42
while True:
    guess=int(input("Guess the number"))
    if guess==secret:
        print("You guessed it right.")
        break
```

```
Guess the number67
Guess the number89
Guess the number42
You guessed it right.
```


Python Literals

- A fixed value that is represented directly in python source code
- Literals are constants that are self-explanatory and don't need to be computed or evaluated.
- They are used to provide values to variable or to directly utilize them in expressions

Python

```
123      # Integer literal
3.1415   # Floating-point literal
"Hello, world!" # String literal
True     # Boolean literal
None     # None literal
```

- Numeric literals: Integers, floating-point numbers, and complex numbers.
- String literals: A sequence of characters enclosed in quotation marks.
- Boolean literals: `True` and `False`.
- None literal: A special value that represents the absence of a value.

References / Resources/ Acknowledgments

- Starting out with Python , 3rd Edition
 - Chapter 3 Decision Structure and Boolean Logic
 - Chapter 4 Repetition Structures
- <https://code4coding.com>
- <https://www.geeksforgeeks.org/python-while-loop/>